Technical Report

CPU Scheduling Comparison

Priority Scheduling vs. Shortest Remaining Time First (SRTF)

Team Members

Soliman ElSayed Soliman
Abdelrahman Mohamed Abdelrahman
Karim Fady Hassan
Sara Mostafa Salah
Salma Ahmed Saeed
Sara Yasser Mohamed

GitHub Repository

`https://github.com/Abdelrahman-mohamedd/OS-Scheduler`

1.Introduction:

CPU scheduling is a fundamental mechanism in modern operating systems, determining how the CPU is allocated among competing processes. The choice of scheduling algorithm significantly impacts system throughput, response time, and fairness.

This report documents the design, implementation, and evaluation of a web-based tool that simulates and compares two preemptive scheduling algorithms:
•**Priority Scheduling (Preemptive)** where the CPU is always given to the highest-priority ready process.
•**Shortest Remaining Time First (SRTF)** a preemptive variant of SJF that selects the process with the least remaining burst time.

The tool was built as a client-side web application (HTML, CSS, JavaScript) and provides live Gantt charts, computed metrics (WT, TAT, RT), a side-by-side comparison table, and full input validation.

2.System Overview:

2.1 Architecture

The application is entirely client-side, no server or backend is required. The codebase is organized into three files:

•**index.html**: document skeleton and layout
•**style.css**: responsive grid, color theming, Gantt chart styling
•**script.js**: all scheduling logic, metric calculation, DOM manipulation, and input validation

Within script.js the responsibilities are separated by function groups: process data management, Priority scheduler, SRTF scheduler, metric calculator, Gantt renderer, comparison table generator, and input validator.

2.2Technology Stack

Technology	Role
HTML	Page structure, process entry form, results containers
CSS	Responsive layout, Gantt bars, colour coding
JavaScript	Scheduling algorithms, metrics, DOM updates, validation
GitHub Pages	Hosting and public access

3.Algorithm Descriptions:

3.1Priority Scheduling (Preemptive)

Priority Scheduling assigns each process to a numeric priority. At every scheduling decision point (process arrival or completion), the ready queue is re-evaluated, and the CPU is given to the process with the lowest priority number (highest urgency).

Core rules implemented:

•At time 0, if multiple processes arrive simultaneously, the one with the lowest priority number runs first.

•When a new process arrives with a strictly higher priority than the running process, preemption occurs immediately.

•Tie-breaking: equal priority processes are served in FCFS order (by arrival time).

•A process that is preempted retains its remaining burst time and re-enters the ready queue.

•Processes that never run until completion may suffer starvation if higher-priority processes keep arriving (demonstrated in Scenario 3).

3.2Shortest Remaining Time First (SRTF)

SRTF is the preemptive version of Shortest Job First (SJF). At each unit, the scheduler selects the ready process with the smallest remaining burst time.

Core rules implemented:

•When a new process arrives, its remaining time is compared against the currently running process; preemption occurs if the new process is shorter.

•Tie-breaking: equal remaining times are resolved by earliest arrival time (FCFS order).

•The algorithm is optimal for minimizing average waiting time among all online scheduling algorithms.

•Response time for a newly arrived short process is minimized; it gets the CPU almost immediately.

•Long processes may still experience starvation if short processes arrive continuously.

4.Performance Metrics:

All three metrics are computed per process and average across all processes in the workload:

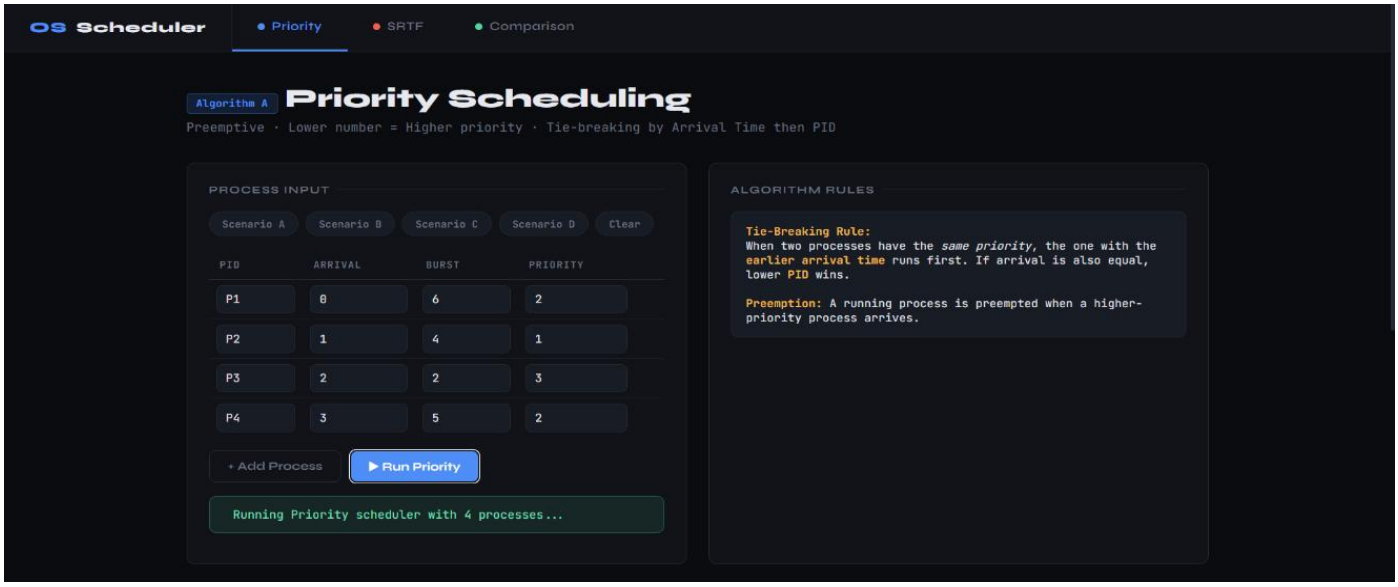
Meaning	Formula	Metric
Waiting Time (WT)	**WT = TAT − Burst Time**	**Time spent in the ready queue, waiting for the CPU.**
Turnaround Time (TAT)	**TAT = Finish Time − Arrival Time**	**Total time from submission to completion.**
Response Time (RT)	**RT = First CPU Time − Arrival Time**	**Time until a process first receives the CPU.**

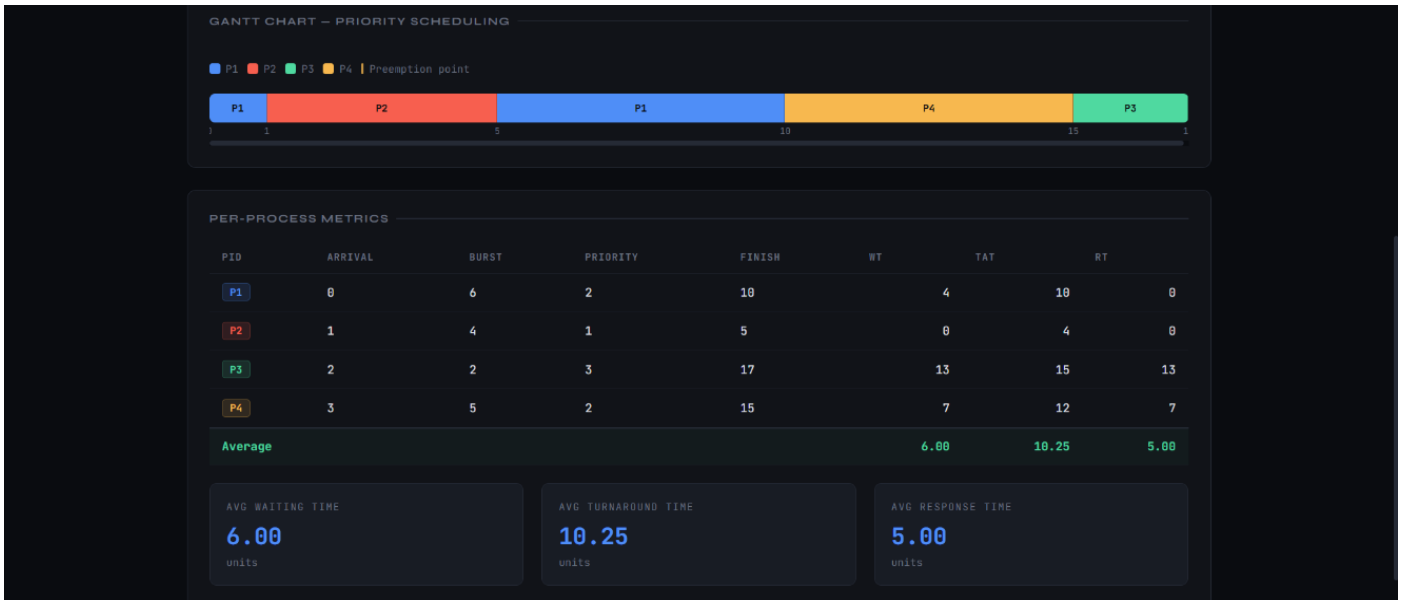
5.Test Scenarios:

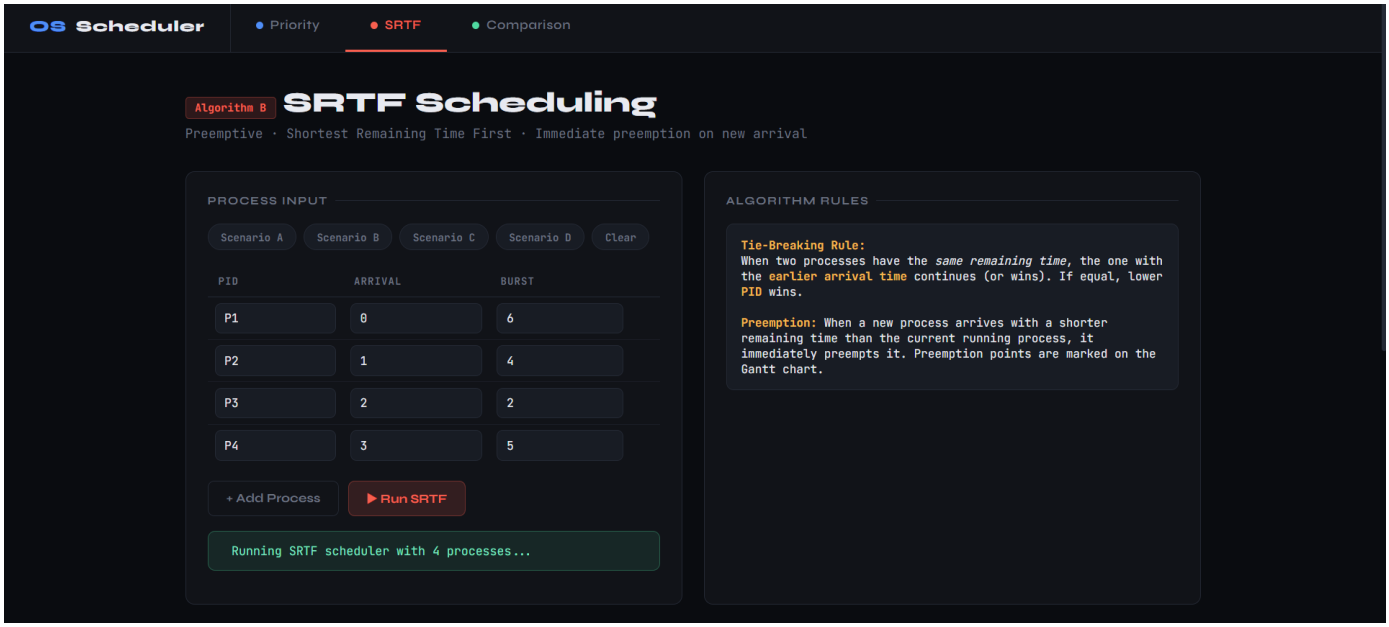
Four distinct scenarios are documented:

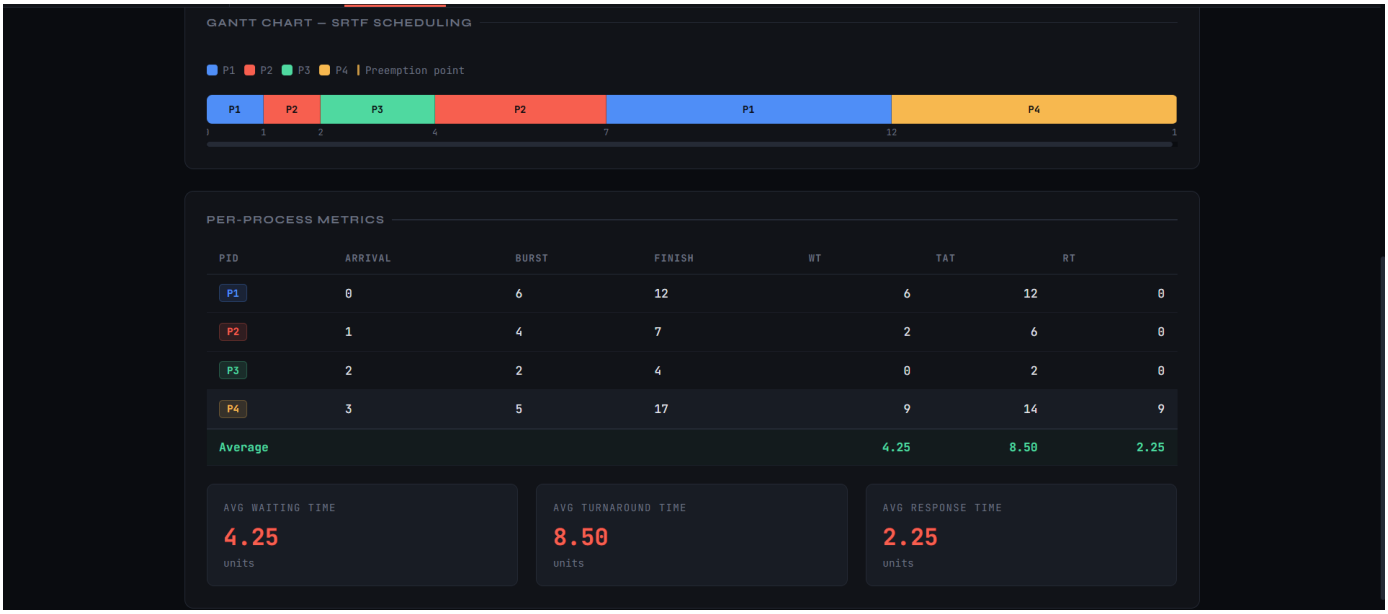
5.1 Scenario A — Normal Mixed Workload:

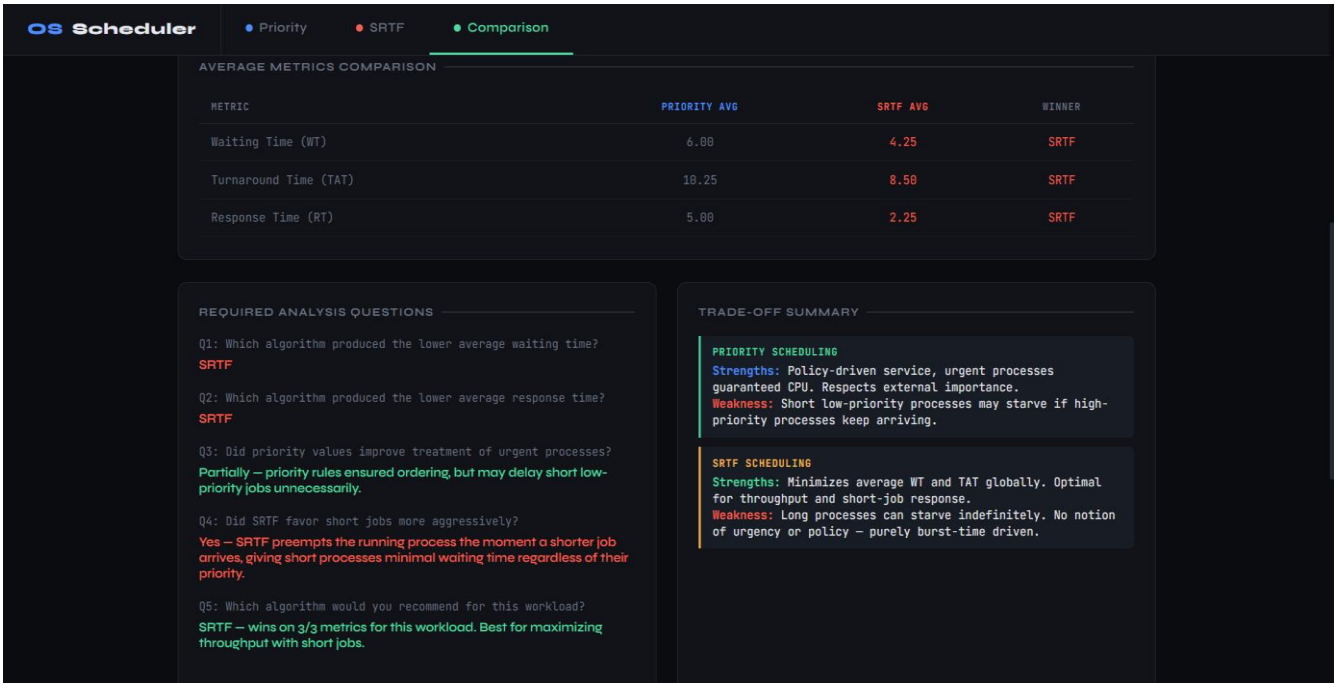
Purpose: Verify that both algorithms handle a standard workload with varied arrival times, burst times, and priorities correctly.

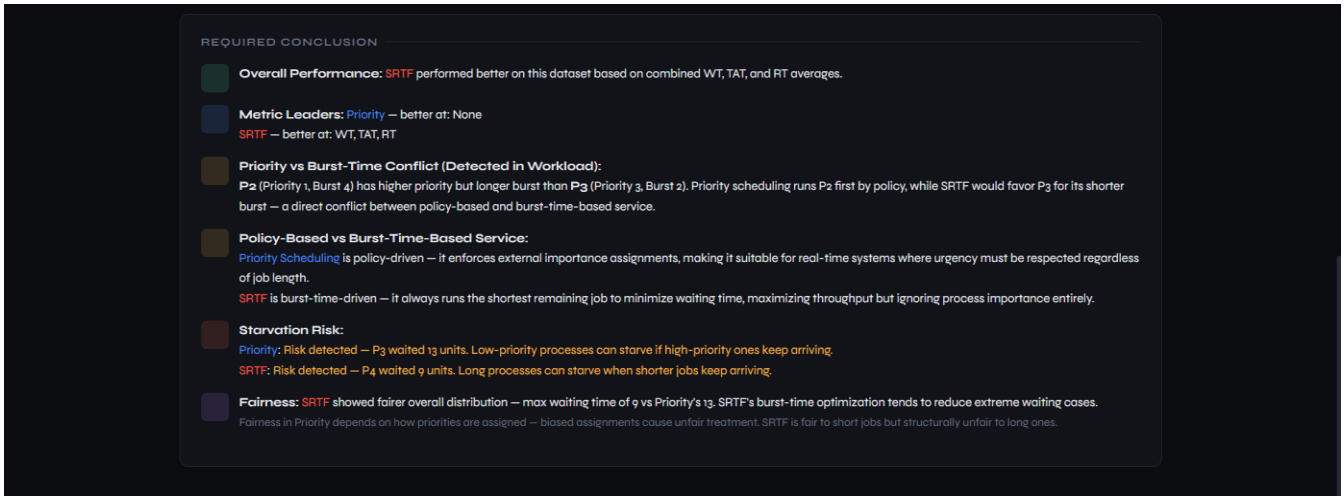





OS Scheduler
Priority
SRTF
Comparison
Algorithm B
SRTF Scheduling
Preemptive · Shortest Remaining Time First · Immediate preemption on new arrival
PROCESS INPUT
Scenario A
Scenario B
Scenario C
Scenario D
Clear
PID
ARRIVAL
BURST
P1
0
6
P2
1
4
P3
2
2
P4
3
5
+ Add Process
▶ Run SRTF
Running SRTF scheduler with 4 processes...
ALGORITHM RULES
Tie-Breaking Rule:
When two processes have the same remaining time, the one with the earlier arrival time continues (or wins). If equal, lower PID wins.
Preemption: When a new process arrives with a shorter remaining time than the current running process, it immediately preempts it. Preemption points are marked on the Gantt chart.


GANTT CHART — SRTF SCHEDULING
P1
P2
P3
P4
Preemption point
P1
P2
P3
P2
P1
P4
0
1
2
4
7
12
16
PER-PROCESS METRICS
PID
ARRIVAL
BURST
FINISH
WT
TAT
RT
P1
0
6
12
6
12
0
P2
1
4
7
2
6
0
P3
2
2
4
0
2
0
P4
3
5
17
9
14
9
Average
4.25
8.50
2.25
AVG WAITING TIME
4.25
units
AVG TURNAROUND TIME
8.50
units
AVG RESPONSE TIME
2.25
units

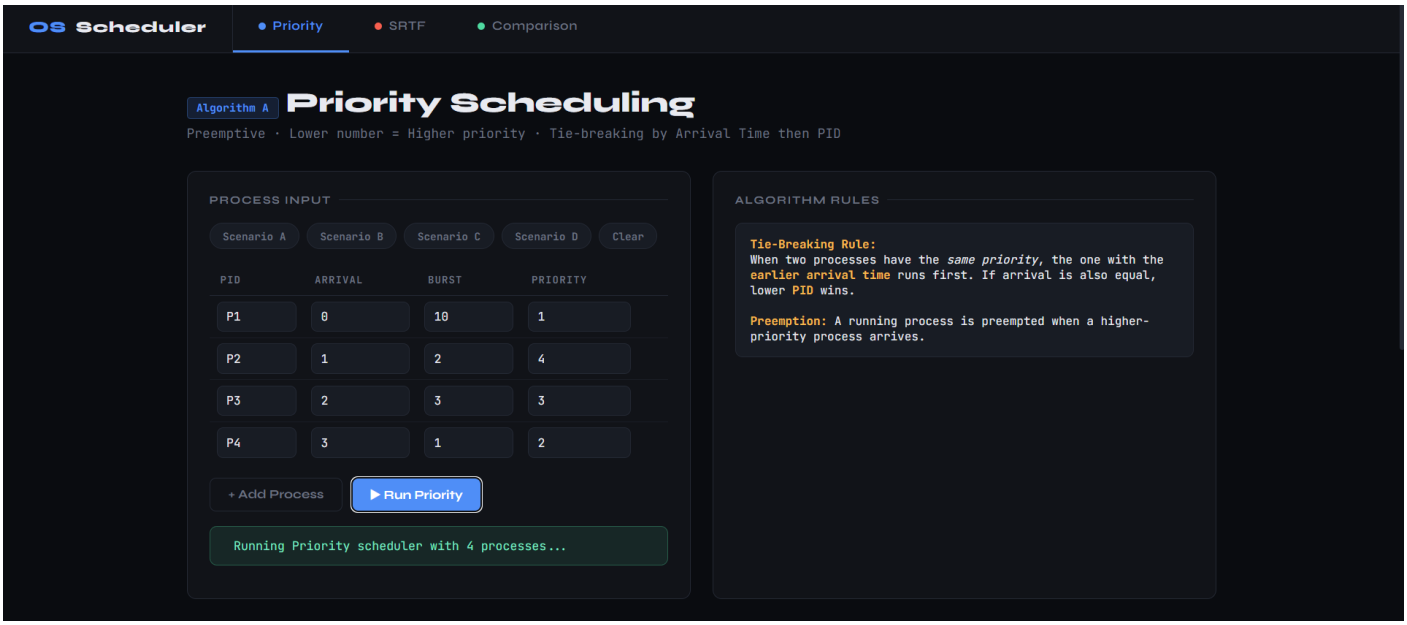


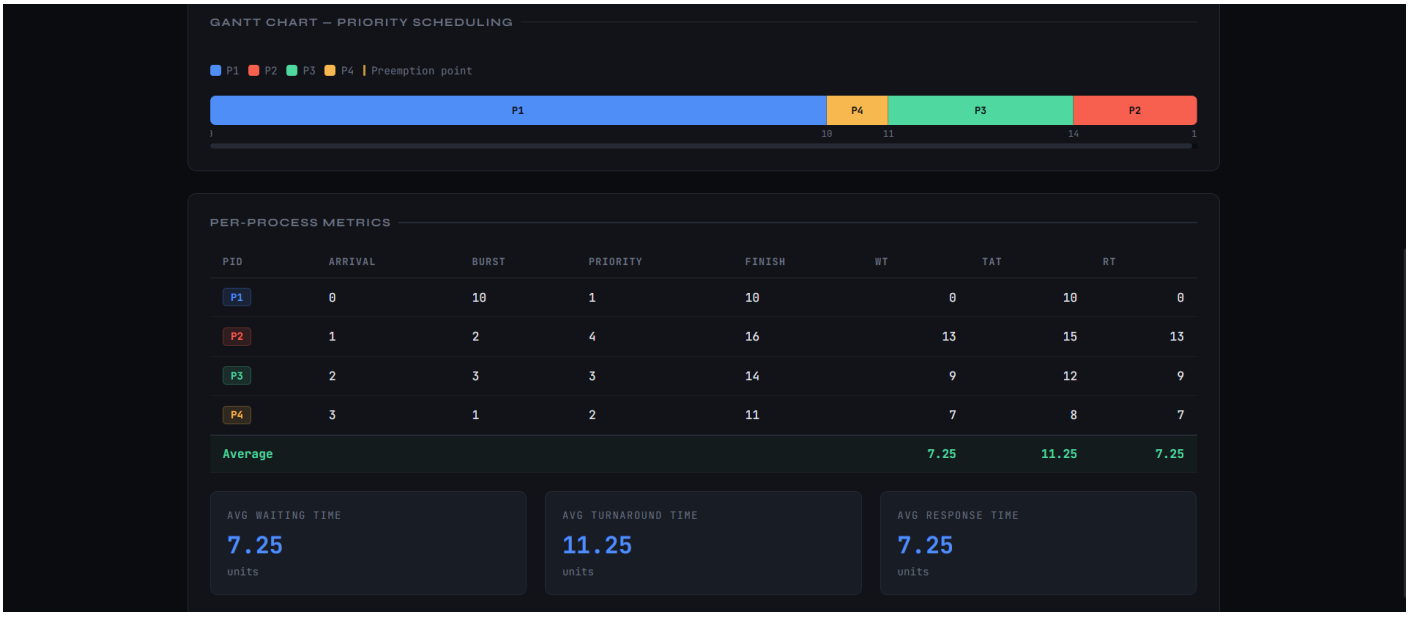


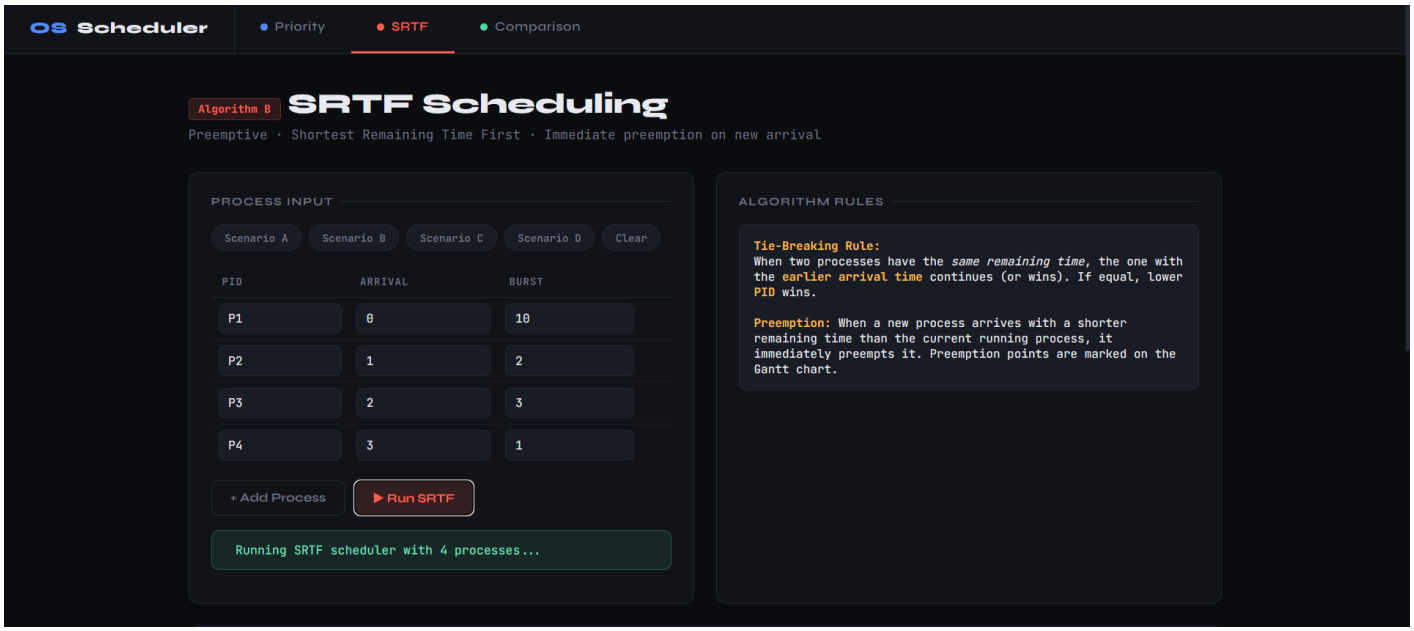
Observation: In this scenario SRTF achieves a lower average waiting time by leveraging short processes (P3) earlier. Priority scheduling keeps the high-priority P2 ahead but at the cost of delaying P3.

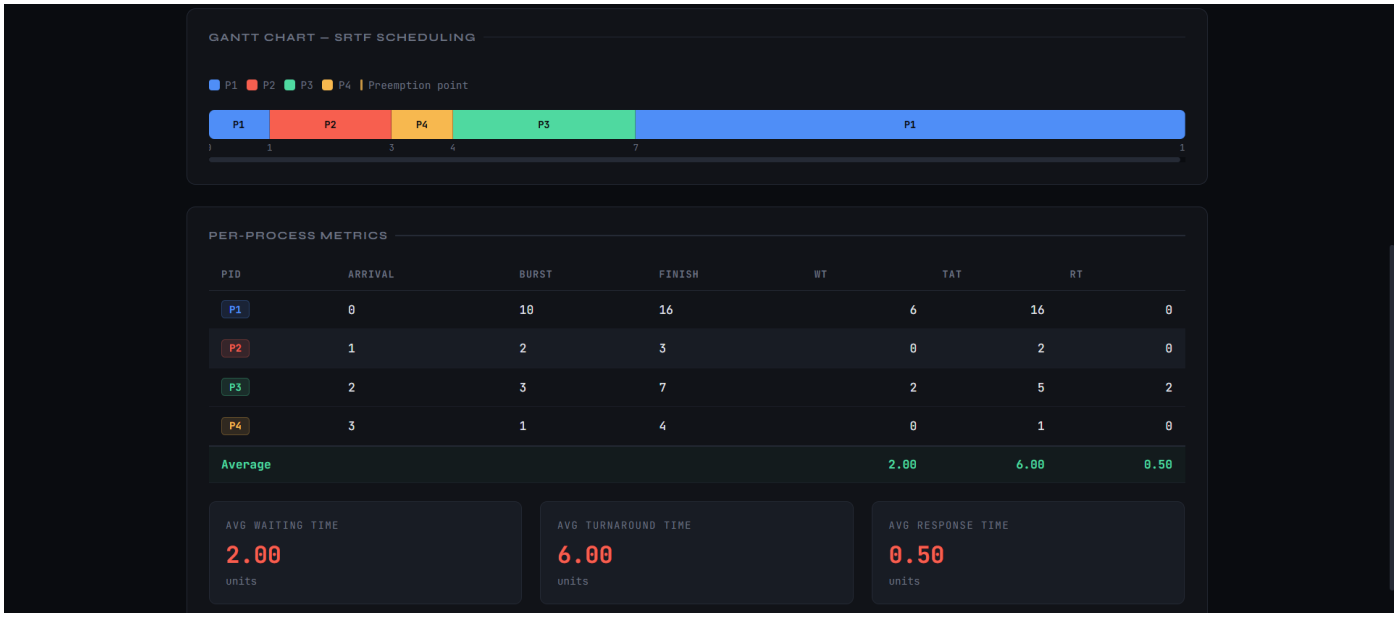
5.2 Scenario B — Priority vs. Burst Time Conflict:

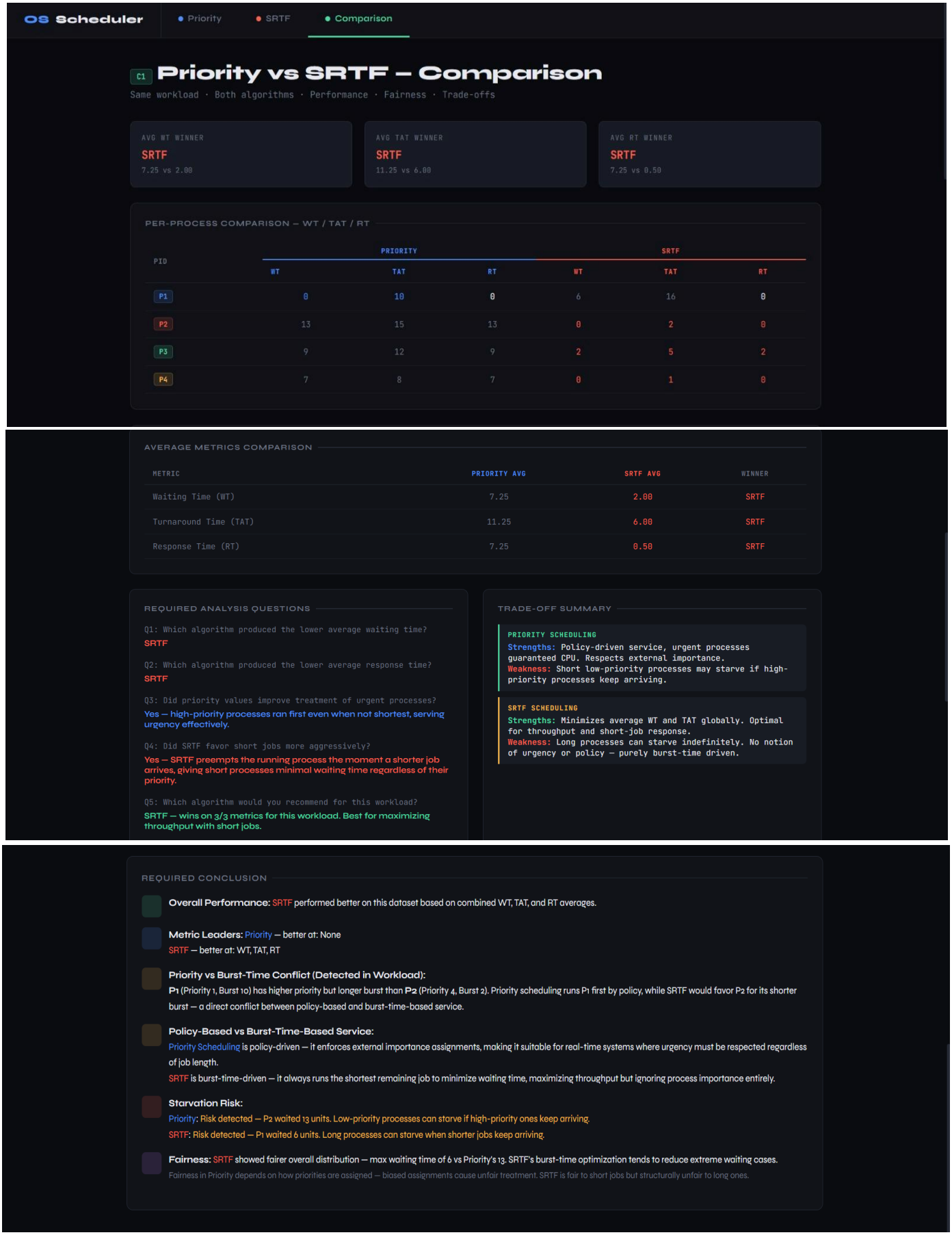
Purpose: Highlight the key behavioral difference — Priority Scheduling honors importance regardless of length, while SRTF optimizes for shortest execution time.






OS Scheduler
Priority
SRTF
Comparison
Algorithm B
SRTF Scheduling
Preemptive · Shortest Remaining Time First · Immediate preemption on new arrival
PROCESS INPUT
Scenario A
Scenario B
Scenario C
Scenario D
Clear
PID
ARRIVAL
BURST
P1
0
10
P2
1
2
P3
2
3
P4
3
1
+ Add Process
▶ Run SRTF
Running SRTF scheduler with 4 processes...
ALGORITHM RULES
Tie-Breaking Rule:
When two processes have the same remaining time, the one with the earlier arrival time continues (or wins). If equal, lower PID wins.
Preemption: When a new process arrives with a shorter remaining time than the current running process, it immediately preempts it. Preemption points are marked on the Gantt chart.

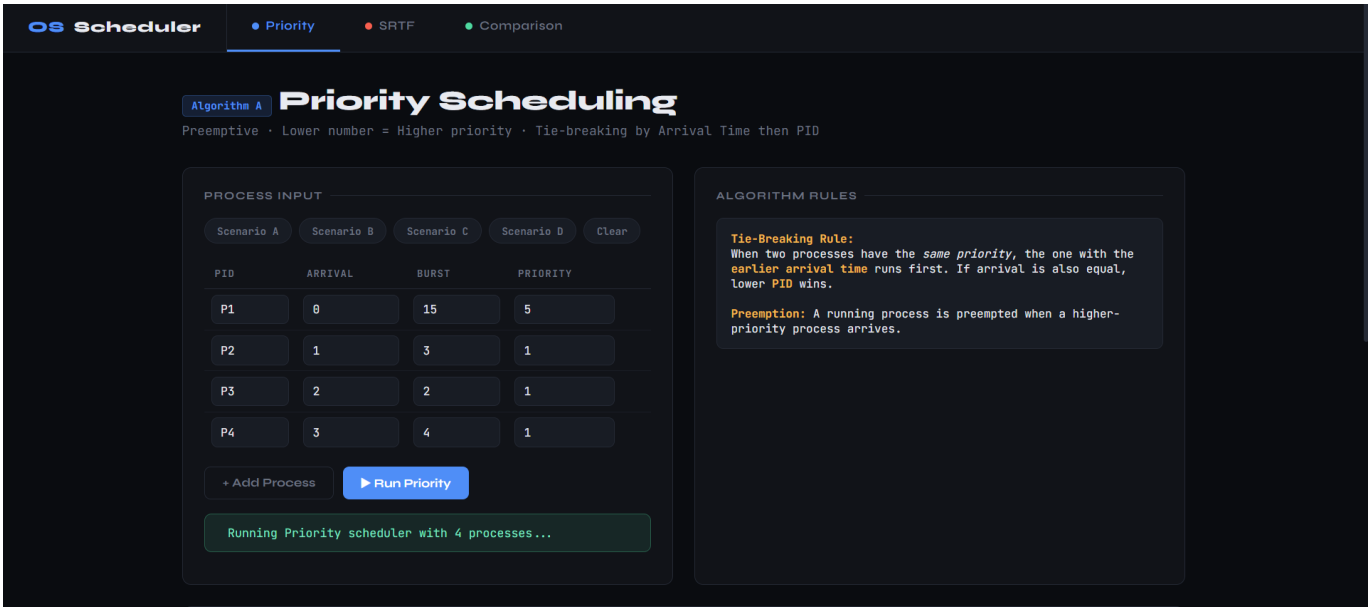

GANTT CHART — SRTF SCHEDULING
P1
P2
P3
P4
Preemption point
P1
P2
P4
P3
P1
1
3
4
7
PER-PROCESS METRICS
PID
ARRIVAL
BURST
FINISH
WT
TAT
RT
P1
0
10
16
6
16
0
P2
1
2
3
0
2
0
P3
2
3
7
2
5
2
P4
3
1
4
0
1
0
Average
2.00
6.00
0.50
AVG WAITING TIME
2.00
units
AVG TURNAROUND TIME
6.00
units
AVG RESPONSE TIME
0.50
units

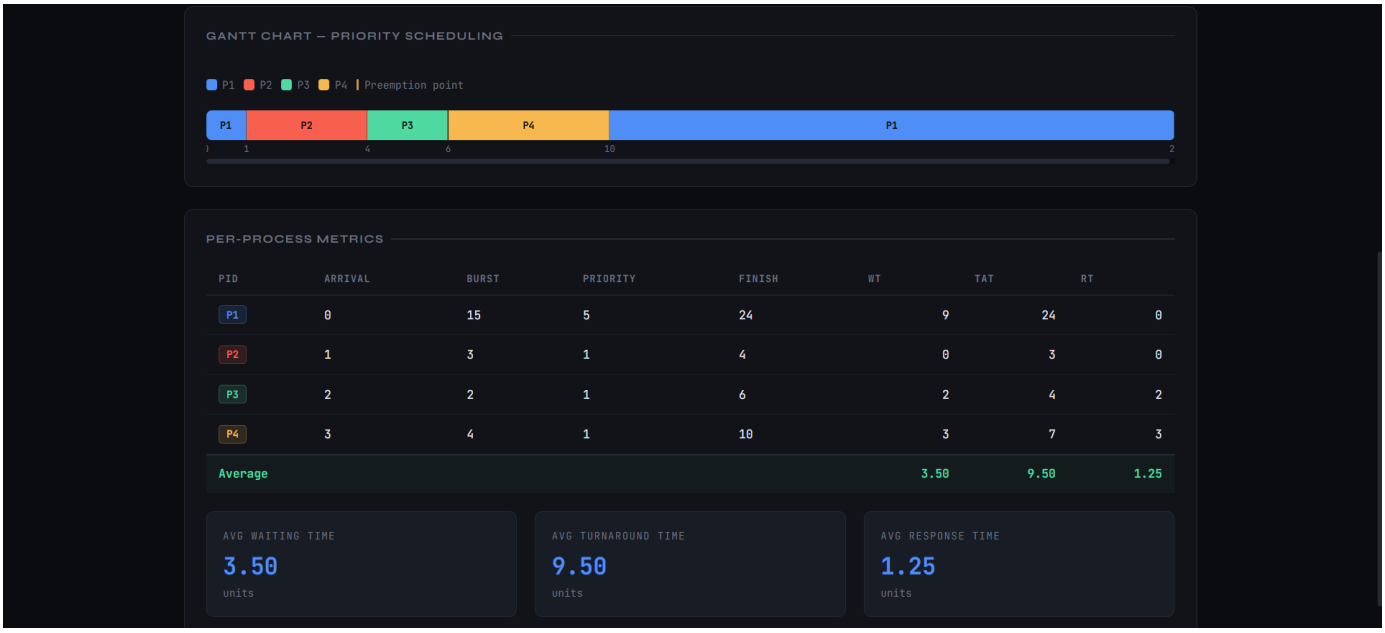


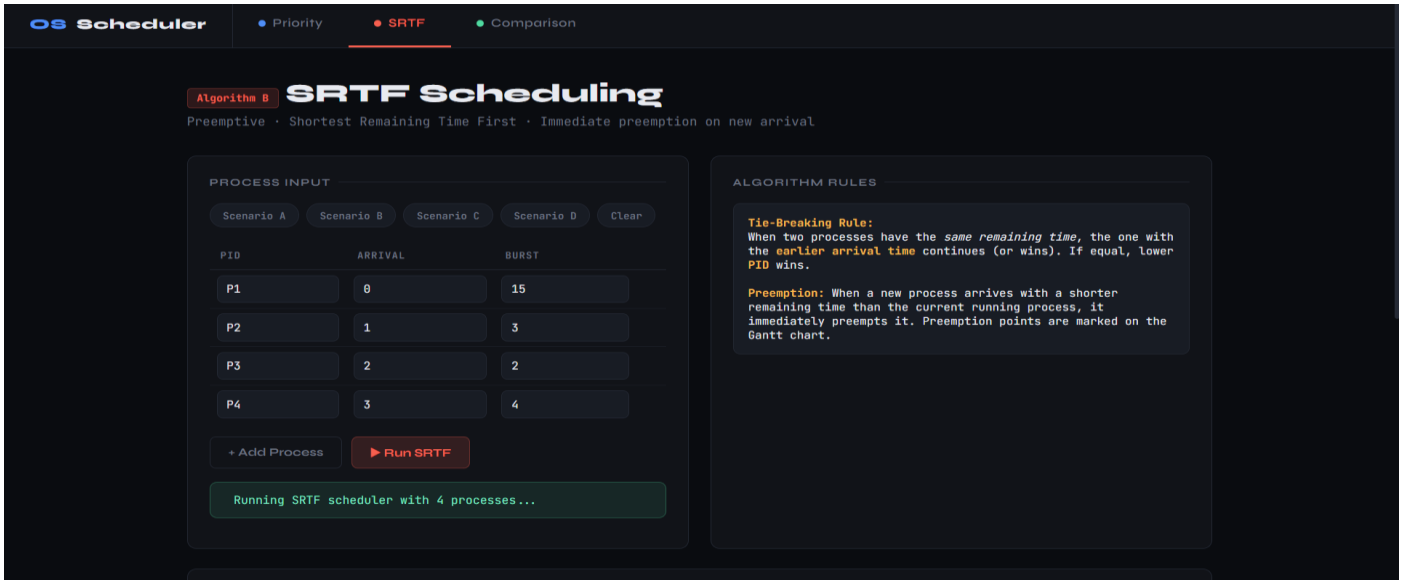
Observation: P1 (highest priority, longest burst) monopolizes the CPU under Priority Scheduling until higher-priority short processes arrive. Under SRTF, P2 and P4 preempt immediately, leading to significantly lower average WT.

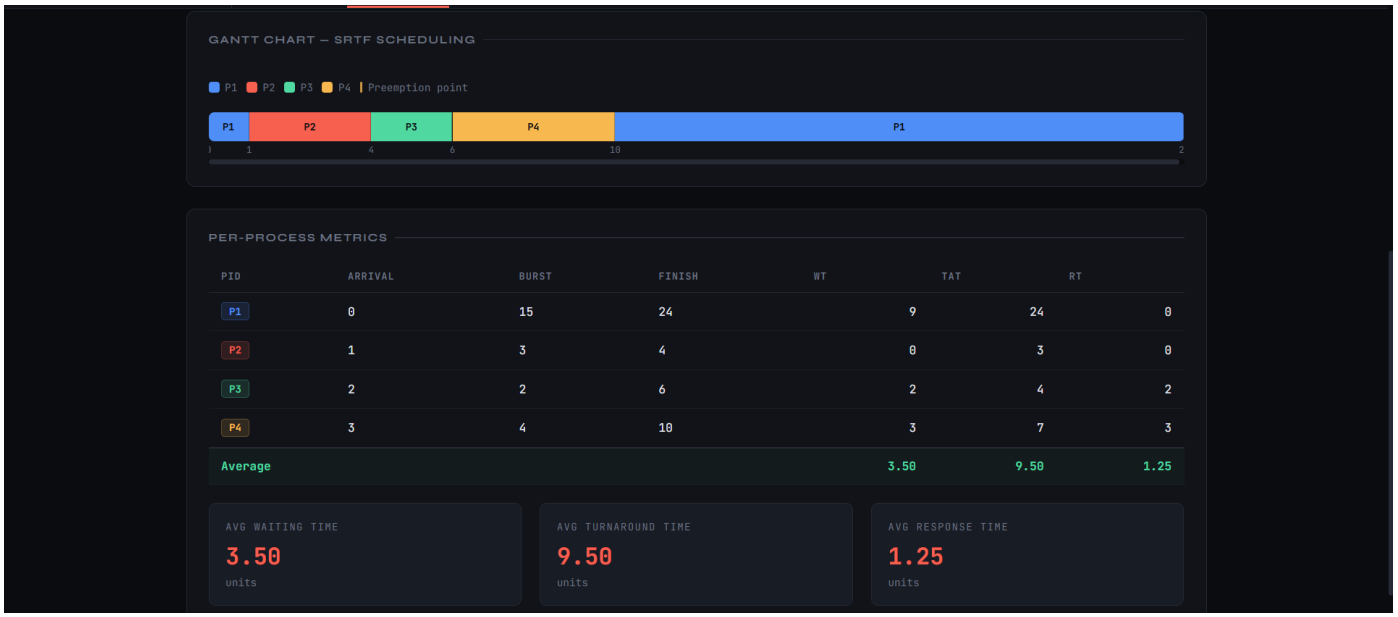
5.3 Scenario C — Starvation Demonstration:

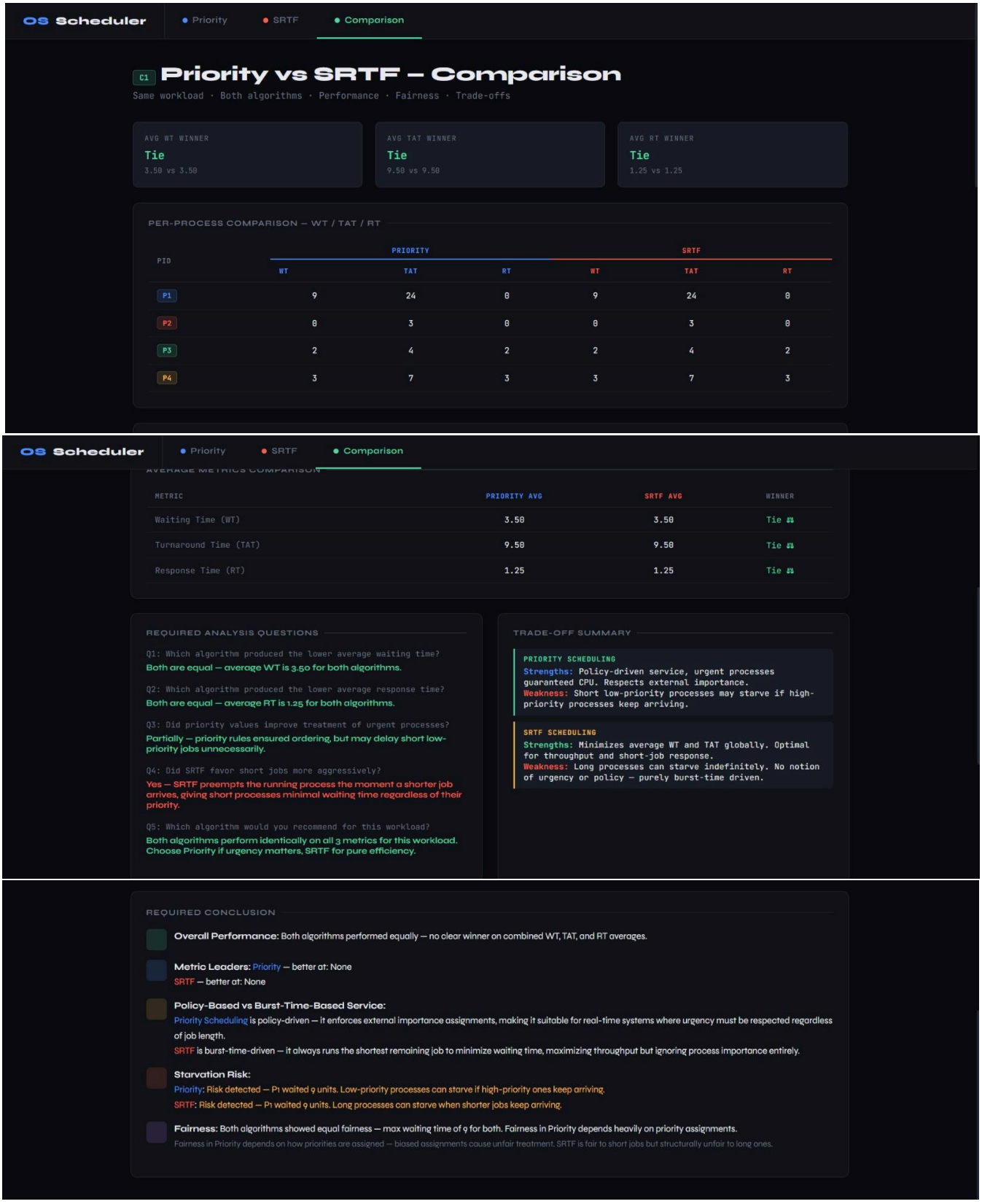
Purpose: Show how continuous arrival of high-priority (or short) processes can cause starvation for a long low-priority process.






OS Scheduler
Priority
SRTF
Comparison
Algorithm B
SRTF Scheduling
Preemptive · Shortest Remaining Time First · Immediate preemption on new arrival
PROCESS INPUT
Scenario A
Scenario B
Scenario C
Scenario D
Clear
PID
ARRIVAL
BURST
P1
0
15
P2
1
3
P3
2
2
P4
3
4
+ Add Process
▶ Run SRTF
Running SRTF scheduler with 4 processes...
ALGORITHM RULES
Tie-Breaking Rule:
When two processes have the same remaining time, the one with the earlier arrival time continues (or wins). If equal, lower PID wins.
Preemption: When a new process arrives with a shorter remaining time than the current running process, it immediately preempts it. Preemption points are marked on the Gantt chart.

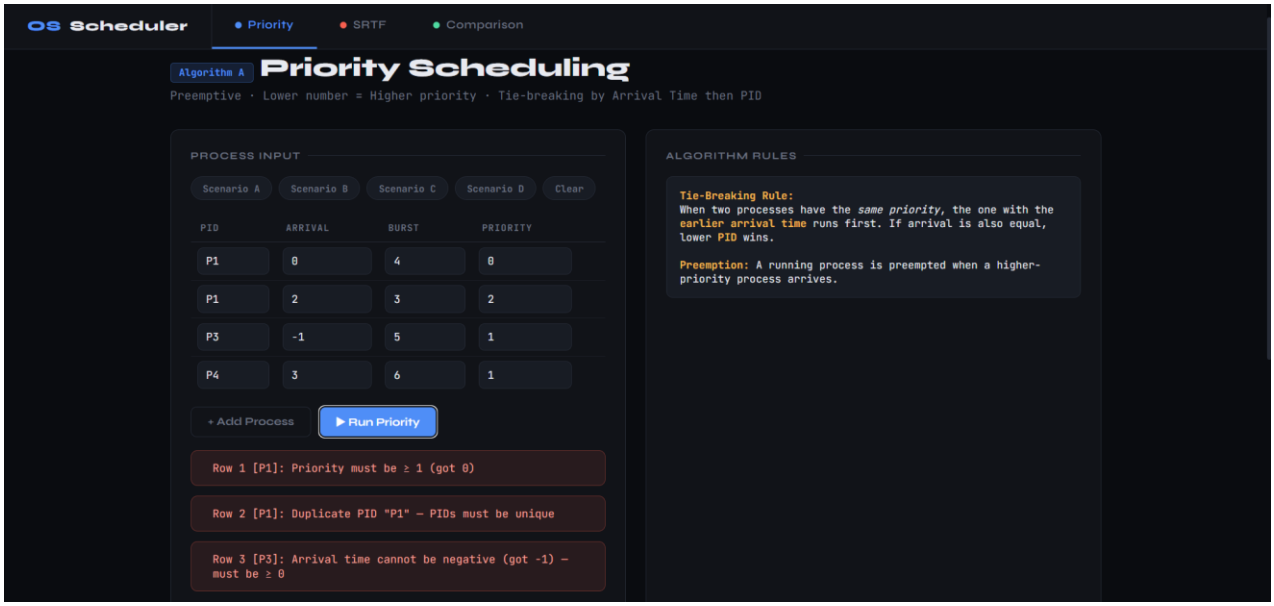

GANTT CHART — SRTF SCHEDULING
P1
P2
P3
P4
Preemption point
P1
P2
P3
P4
P1
1
4
6
10
PER-PROCESS METRICS
PID
ARRIVAL
BURST
FINISH
WT
TAT
RT
P1
0
15
24
9
24
0
P2
1
3
4
0
3
0
P3
2
2
6
2
4
2
P4
3
4
10
3
7
3
Average
3.50
9.50
1.25
AVG WAITING TIME
3.50
units
AVG TURNAROUND TIME
9.50
units
AVG RESPONSE TIME
1.25
units

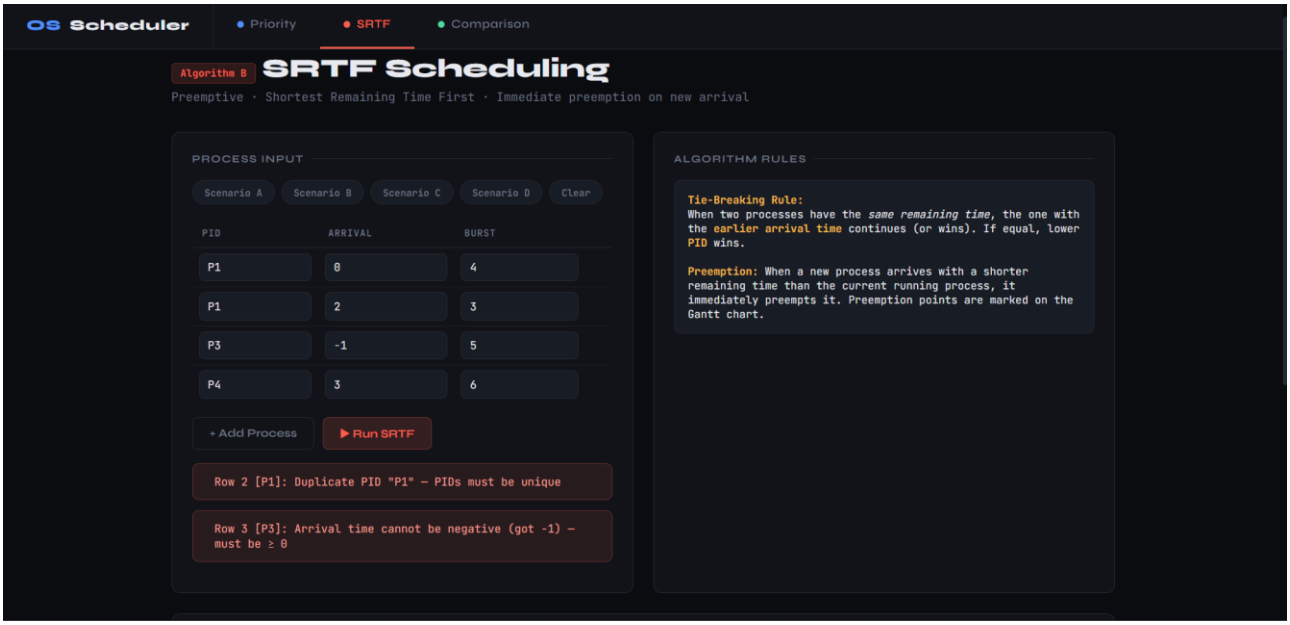


Observation: P1 (priority 5 — lowest urgency) waits the longest under Priority Scheduling because P2, P3, and P4 have higher priority and preempt it repeatedly. SRTF also delays P1 significantly because its remaining time is always the largest, illustrating that starvation is a risk in both algorithms when process patterns are extreme.

5.4 Scenario D — Invalid Input Validation:

Purpose: This scenario tests the system's input validation by providing invalid data. It verifies that the scheduler correctly detects and reports errors before running.





Result: The application detected **all 3 validation errors** in the submitted input, including **duplicate Process ID**, **negative arrival time**, and **invalid zero priority**, and **blocked the simulation from running**. No **Gantt chart** was rendered and no **metrics** were calculated, confirming that the **validation layer** correctly prevents erroneous scheduling results caused by malformed data.

6.Assumptions and Limitations:

6.1 Assumptions:

•All processes are CPU-bound (no I/O bursts included).

•Burst times are known in advance — a common assumption for scheduling simulations.

•Context-switch overhead is assumed to be zero

•Time is modelled as discrete integer units.

•Priority numbers: lower number = higher priority

•When **two processes have equal priority** (in Priority Scheduling) or equal remaining time (in SRTF), FCFS order (earliest arrival) is used as the tiebreaker

---

6.2 Limitations:

•Burst time estimation: SRTF requires knowing burst times in advance, which is not always possible in real systems.

•No, I/O modelling: The simulator does not model processes that alternate between CPU and I/O bursts. A multi-queue or multilevel feedback queue model would be needed for that.

•No aging mechanism: Priority Scheduling does not implement aging (gradually increasing the priority of waiting processes), so starvation remains possible.

•The simulator works with one CPU only.

•No memory or resource constraints are modelled.

7.Conclusion:

This project successfully implements and visualizes two preemptive CPU scheduling algorithms—**Priority Scheduling** and **SRTF**—through an interactive web based tool. The simulation demonstrates:

• Correct process selection, pre-emption, and completion behaviour for both algorithms.
• Accurate computation of Waiting Time, Turnaround Time, and Response Time for each process, as well as their averages.
• A clear and readable Gantt chart visualization with time markers for each algorithm.
• Four well-designed test scenarios that highlight normal operation, algorithmic trade-offs, and starvation behaviour.
• Robust input validation that prevents invalid or inconsistent data from reaching the scheduler.

From the comparative analysis, SRTF proves to be the superior choice for throughput-oriented workloads with similar process importance, while Priority Scheduling remains indispensable when task criticality must be respected. Understanding when to apply each algorithm—and recognizing their respective limitations—is a core competency in operating systems design.